

Appendix B

Answers to Questions

Chapter 1, An Overview

1.1. What are the three parts of a user story?

Answer: Card, Conversation, and Confirmation.

1.2. Who is on the customer team?

Answer: The customer team includes those who ensure that the software will meet the needs of its intended users. This may include testers, a product manager, real users, and interaction designers.

1.3. Which of the following are not good stories? Why?

Answer: See Table B.1.

Table B.1 *Answer to Question 1.3.*

Story	Answer
a. The user can run the system on Windows XP and Linux.	This is a good story.
b. All graphing and charting will be done using a third-party library.	This is not a good story. Users will not care how the graphing and charting are done.
c. The user can undo up to fifty commands.	This is a good story.
d. The software will be released by June 30.	This is not a good story. It is a constraint that will need to be considered during release planning.
e. The software will be written in Java.	This is probably not a good story but it depends on the product. If the product is a class library being marketed to Java programmers, those users will care about the language.

Table B.1 Answer to Question 1.3. (Continued)

Story	Answer
f. The user can select her country from a drop-down list.	This is a good story but it may be a little small.
g. The system will use Log4J to log all error messages to a file.	This is not a good story as written. It should not specify that Log4J be used as the logging mechanism.
h. The user will be prompted to save her work if she hasn't saved it for 15 minutes.	This is a good story.
i. The user can select an "Export to XML" feature.	This is a good story.
j. The user can export data to XML.	This is a good story.

1.4. What advantages do requirements conversations have over requirements documents?

Answer: Written documents imply a precision that they can't back. User stories, with cards as reminders to hold conversations, avoid the false appearance of being highly precise. Writing things down is no guarantee that customers will get what they want; at best customers will get what was written down. Frequent conversations, especially ones close to and during development of the feature being discussed, lead to greater and shared understanding between the developers and customers.

1.5. Why would you want to write tests on the back of a story card?

Answer: Writing tests on the back of a card is a great way for the customer to communicate her expectations and assumptions about a story.

Chapter 2, Writing Stories

2.1. For the following stories, indicate if it is a good story or not. If not, why?

Answer: See Table B.2.

2.2. Break this epic up into appropriately sized component stories: "A user can make and change automated job search agents."

Answer: Minimally this epic should be split into two stories, one to make and one to change agents. However, it could be split many different ways, depending on how long the stories are likely to take to implement. One possible disaggregation is:

- A user can make an automated job search agent.

- A user can edit the search parameters of an automated job search agent.
- A user can change the times when an automated job search agent will run.
- A user can change how the results of an automated job search agent will be reported.

Table B.2 Answer to Question 2.1.

Story	Answer
a. A user can quickly master the system.	This story should be changed. Neither “quickly” nor “master” is defined.
b. A user can edit the address on a resume.	This story is probably too small, but it could be OK depending on how long it would likely take the developers to implement.
c. A user can add, edit and delete multiple resumes.	This is a compound story and should be split into multiple stories.
d. The system can calculate saddlepoint approximations for distributions of quadratic forms in normal variables.	If the customer wrote this story then she probably knows what it means. However, if the developers don’t understand this story then the customer should consider rewriting it (or at least having a good conversation about it) so that the developers can estimate it.
e. All runtime errors are logged in a consistent manner.	This story is fine as is.

Chapter 3, User Role Modeling

3.1. Take a look at the eBay website. What user roles can you identify?

Answer: Your list should include some roles similar to these: One-time Seller, Small Seller, Frequent Seller, Infrequent Buyer, Frequent Buyer, Corporate Seller, Manufacturer, Payment Processor, Collector, Club Member, Software Developer, Affiliate, Wireless Seller, Wireless Buyer

3.2. Consolidate the roles you came up with in the previous question and show how you would lay out the role cards. Explain your answer.

Answer: From my list, I consolidated sellers into a generic Seller role and three more specialized sellers: Small Seller, Frequent Seller and

Corporate Seller. Similarly, I had a generic Buyer role specialized into Infrequent Buyer, Frequent Buyer and Collector. I also kept Payment Processor, Affiliate and a generic Wireless User.

- 3.3. Write persona descriptions for the one most important user role.

Answer: Brenda is a Frequent Buyer. During a typical week she visits the site at least once a day, and makes an average of one or two purchases per week. She typically buys movies and books but she has also purchased gardening and kitchen items. She is a realtor and is very comfortable on our site but is a little uncomfortable learning most new software. She usually accesses the site from her dial-up connection at home but occasionally accesses it from her faster office connection.

Chapter 4, Gathering Stories

- 4.1. What problems would you expect if a team only gathered requirements through the use of questionnaires?

Answer: Questionnaires can take a long time to turn around so the project will take longer to complete. Someone will need to aggregate and interpret the results, which means there will be some degree of misinterpretation. Because questionnaires do not provide for true two-way communication, it will be extremely hard for a team to get feedback as to whether it's on the right track.

- 4.2. Rephrase the following questions to be context free and open-ended. Do you think the user should have to enter a password? Should the system automatically save the user's work every 15 minutes? Can one user see database entries saved by another user?

Answer: There are many ways to reword these. Here are some examples:

- Describe how the system should safeguard sensitive data.
- What would a user do if the system crashed while she was using it?
- Tell me about the accessibility of data saved by a user.

- 4.3. Why is it best to ask open-ended, context-free questions?

Answer: Context-free questions do not imply an answer ("When did you stop beating your wife?") so the respondent does not feel a need to give the "right" answer. Open-ended questions allow detailed responses that go beyond a simple yes or no. Open-ended, context-free questions are best because they do not influence the

response and they allow a broader range of responses than yes or no.

Chapter 5, Working with User Proxies

- 5.1. What problems can result from using the users' manager as a proxy for the users?

Answer: Even if the users' manager is a current user of the software, her needs almost certainly differ from the needs of the users. Worse, if she is a former user, her knowledge of the system is outdated.

- 5.2. What problems can result from using a domain expert as a proxy for users?

Answer: One problem is that the domain expert may not be a user of the system. If she is, her use of the system may differ from less expert users. A second problem is that you may end up with a system that is perfect for experts, but that is not usable by those with less than expert level domain knowledge.

Chapter 6, Acceptance Testing User Stories

- 6.1. Who specifies the tests? Who helps?

Answer: The customer specifies the tests. The customer will usually work with a programmer or tester to actually create the tests, but minimally the customer needs to specify the tests that will be used to know when a story has been correctly developed.

- 6.2. Why specify tests before the stories are coded?

Answer: Tests are specified before coding begins because they are a useful and effective method for communicating the customer's assumptions about the new functionality.

Chapter 7, Guidelines for Good Stories

- 7.1. Assume the story "A Job Seeker can search for open jobs" is too large to fit into one iteration. How would you split it?

Answer: This story can probably be split based on the search parameters supported such as location, key words, job title, salary, and so on. Additionally, there may be different ways to present the results.

An initial story might cover a very simple list of each matched job. That story could be augmented by a story to enhance the display of the results, perhaps with more details about each job, allowing the user to select a sort order or what fields are displayed, or providing a link to more details about the job.

7.2. Which of these stories is appropriately sized and can be considered a closed story?

Answer: See Table B.3.

Table B.3 *Answer to question 7.2.*

Story	Answer
a. A user can save her preferences.	This story may or may not be closed, depending on the system. If saving preferences is something a user may want to do, it can probably be considered closed. It might be on the small side but is probably OK, again depending on the system and the team building it.
b. A user can change the default credit card used for purchases.	This is a closed and appropriately sized story.
c. A user can log on to the system.	This story is not closed and is probably too small.

7.3. What simple changes could improve the story “Users can post their resumes”?

Answer: As written, it is not clear if a user can post multiple resumes. This undoubtedly will come out during conversations about the story, but it is better to write the story more clearly as “A Job Seeker can post one or more resumes.”

7.4. How would you test the constraint “The software will be easy to use”?

Answer: In order to test this, you must first define what “easy to use” means. Does it mean that a skilled user can complete common tasks with a minimum number of keystrokes? Or does it mean that a new user can quickly achieve a given level of proficiency with the software? Most commonly it will mean the latter. If so, define one or more tests such as:

- A new user can search for a job, register on the system, and post her resume within 30 minutes of first seeing the system.

Tests like these can’t be performed as part of a project’s nightly builds, but they can be verified by occasional usability testing during which new users are shown the software and observed.

Chapter 8, Estimating User Stories

- 8.1. During an estimating meeting three programmers are estimating a story.

Individually they estimate the story at two, four and five story points.

Which estimate should they use?

Answer: They should continue discussing the story until their estimates get closer.

- 8.2. What is the purpose of triangulating estimates?

Answer: Triangulation improves estimates by making sure that each estimate makes sense in relation to multiple other estimates. If a two-point story seems to be twice a one-point story, it should also seem to be one-half of a four-point story.

- 8.3. Define velocity.

Answer: Velocity is the number of story points completed by a team in an iteration.

- 8.4. Team A finished 43 story points in their last two-week iteration. Team B is working on a separate project and has twice as many developers. They also completed 43 story points in their last two-week iteration. How can that be?

Answer: The story points of one team are not comparable to the story points of any other team. From the information in this question, we cannot infer that Team A is twice as productive as Team B.

Chapter 9, Planning a Release

- 9.1. What are three ways of estimating a team's initial velocity?

Answer: You can use historical values, take a guess, or run an initial iteration and use the velocity of that iteration.

- 9.2. Assuming one-week iterations and a team of four developers, how many iterations will it take the team to complete a project with 27 story points if they have a velocity of 4?

Answer: With a velocity of 4 and 27 story points in the project, it will take the team 7 iterations to finish.

Chapter 10, Planning an Iteration

10.1. Disaggregate this story into its constituent tasks: A user can view detailed information about a hotel.

Answer: There are, of course, many ways to do this, but here's one:

- Design the look of these web pages.
- Code the HTML to display hotel and room photos.
- Code the HTML to display a map showing where the hotel is.
- Code the HTML to display a list of hotel amenities and services.
- Figure out how we're generating maps.
- Write SQL to retrieve information from the database.
- And so on.

Chapter 11, Measuring and Monitoring Velocity

11.1. A story estimated at one story point actually took two days to complete. How much does it contribute to velocity when calculated at the end of the iteration?

Answer: It contributes one point.

11.2. What can you learn from a daily burndown chart that you can't see on an iteration burndown chart?

Answer: Daily burndown charts show the progress of the team during the iteration. You can use this information to gauge whether all planned work will be completed by the end of the iteration. If it becomes apparent that not all work can be completed, the team and customer can talk during the iteration about which work should be deferred.

11.3. What conclusions should you draw from Figure 11.7? Does the project look like it will finish ahead, behind or on schedule?

Answer: This team started out a little better than anticipated in the first iteration. They expected velocity to improve in the second and third iterations and then stabilize. After two iterations they have already achieved the velocity they expected after three iterations. At this point they are ahead of schedule but you should be reluctant to draw too many firm conclusions after only two iterations.

11.4. What is the velocity of the team that finished the iteration shown in Table 11.3?

Answer: Sixteen. Partially completed stories do not contribute to velocity.

11.5. What circumstances would cause an iteration burndown chart to reflect an upward trend?

Answer: An iteration burndown chart will trend upwards if new work is being added faster than known work is being completed, or if the team decides that a significant amount of future work has been underestimated.

11.6. Complete Table 11.4 by writing the missing values into the table.

Answer: The completed table appears as Table B.4.

Table B.4 *Answer to question 11.6.*

	Iteration 1	Iteration 2	Iteration 3
Story points at start of iteration	100	76	34
Completed during iteration	35	40	36
Changed estimates	5	-5	0
Story points from new stories	6	3	2
Story points at end of iteration	76	34	0

Chapter 12, What Stories Are Not

12.1. What are the key differences between user stories and use cases?

Answer: A user story usually encompasses a smaller scope than a use case. User stories do not include as much detail as use cases. User stories are not intended to be useful after the iteration in which they are developed; use cases are often intended as permanent artifacts of a project.

12.2. What are the key differences between user stories and IEEE 830 requirements statements?

Answer: IEEE 830-style requirements statements focus on the attributes of the solution, while user stories focus on the user's goals. IEEE 830 requirements specifications encourage teams to write all of the requirements statements up front rather than in an iterative manner, as with user stories. Great care is taken in writing requirements statements to make sure the words convey the proper meaning; user stories acknowledge the superiority of conversations for clarifying details.

12.3. What are the key differences between user stories and interaction design scenarios?

Answer: Interaction design scenarios are much more detailed than user stories, often describing the persona and the context of the system use in great detail. Also, a scenario often describes a broader scope than does a user story.

12.4. For a non-trivial project, why is it impossible to write all the requirements at the start of the project?

Answer: Attempting to write all requirements at the start of a project ignores an important feedback loop. When a system's intended users begin to see and interact with the system, new requirements come to mind.

12.5. What is the advantage to thinking about users' goals rather than on listing the attributes of the software to be built?

Answer: A list of attributes does not give the reader the same overall understanding of a product that stories and conversations do. Also, if our work is driven by a list of product attributes, when we are done the best we can say is that the delivered product possesses the attributes on the list. This is not the same as saying the delivered product meets all of a user's goals.

Chapter 13, Why User Stories?

13.1. What are four good reasons for using user stories to express requirements?

Answer: User stories emphasize verbal communication, are comprehensible by everyone, are the right size for planning, support iterative development, encourage deferring detail, support opportunistic design, encourage participatory design and build up tacit knowledge.

13.2. What can be two drawbacks to using user stories?

Answer: On large projects it can be difficult to keep hundreds or thousands of stories organized; stories may need to be augmented with additional documents for traceability; and, while great at improving tacit knowledge through face-to-face communication, conversations do not scale adequately to entirely replace written documents on large projects.

13.3. What is the key difference between participatory and empirical design?

Answer: In participatory design, the intended users of a system become a part of the team designing the behavior of that system. In empiri-

cal design, the intended users are instead studied or observed by the designers of the software, who then make all of the design decisions.

- 13.4. What is wrong with the requirements statement, “All multipage reports should be numbered”?

Answer: It is unclear what “should be numbered” means. Does this mean that the programmers should code this functionality but don’t have to? Does it mean pages should be numbered if there’s room on the page?

Chapter 14, A Catalog of Story Smells

- 14.1. What should you do if the team is consistently finding it difficult to plan the next iteration?

Answer: There could be other reasons but you should consider whether too many stories are interdependent, or are too small or too large.

- 14.2. What should the team do if they are consistently running out of room to write on the story cards?

Answer: They should use smaller cards to enforce the discipline of keeping details out of the story descriptions.

- 14.3. What could cause the customer to have a difficult time prioritizing stories?

Answer: The stories may be the wrong size (either too large or too small) or the stories may not clearly express value to users or customers.

- 14.4. How do you know if you are splitting too many stories?

Answer: You have to rely on your gut feel. Stories are often and legitimately split because they were intentionally written as epics to start with, or because they are too big to fit into an iteration. If you find yourself frequently splitting stories for other reasons, you may be doing it too often.

Chapter 15, Using Stories with Scrum

- 15.1. Describe the differences between an incremental and an iterative process.

Answer: An iterative process is one that makes progress through successive refinement. An incremental process is one in which software is built and delivered in pieces.

15.2. What is the relationship between the product backlog and the sprint backlog?

Answer: Items are moved from the product backlog to the sprint backlog at the start of a sprint.

15.3. What is meant by a potentially shippable product increment?

Answer: By the end of each sprint a Scrum team is responsible for creating a potentially shippable product increment. This means the software is coded, tested and could be given to users.

15.4. Who is responsible for prioritizing work and for selecting the work the team will perform during a sprint?

Answer: The Product Owner prioritizes the work but the team selects the work they will perform during a sprint. Naturally they are expected to select from among the top priority items.

15.5. What questions are answered by each team member at the daily scrum?

Answer: What did you do yesterday? What will you do today? What is in your way?

Chapter 16, Additional Topics

16.1. How should you handle a requirement for a system to scale up to use by 1,000 concurrent users?

Answer: You should write this as a constraint and then supplement it with appropriate tests. Depending on the system, you may want to start with a test for 100 concurrent users in one iteration and progressively increase that to 1,000 users over a number of iterations.

16.2. Do you prefer to write stories on note cards or in a software system? Defend your answer.

Answer: The low-tech simplicity of note cards make them ideal for many projects. Cards also offer a limited amount of space, which helps keep stories brief. Because they can be easily shuffled about on a table or wall they are ideal for planning. However, a team that is not collocated or has stringent traceability requirements might prefer to work with software.

16.3. What impact does an iterative process have on the user interface of an application?

Answer: Iterative refinement of the system can make it harder for users to learn the system. When menu systems change or features appear in different places, users must relearn they system.

16.4. Give some examples of systems that could benefit from more upfront consideration of the user interface than is typically given on an agile project.

Answer: There can be many examples but here are a few:

- a commercial product that competes in a mature industry primarily through ease of use
- software aimed at novice users
- software that will be used rarely but for intense periods (such as for income tax preparation)
- software for low vision users or users with a movement disorder

This page intentionally left blank

References

Books and Articles

- Adolph, Steve, Paul Bramble, et al. *Patterns for Effective Use Cases*. Reading, Mass.: Addison-Wesley, 2002.
- Antón, Annie I., and Colin Potts. "The Use of Goals to Surface Requirements for Evolving Systems," in Proceedings of the 20th International Conference on Software Engineering (ICSE 98), April 1998: 157–166.
- Astels, Dave. *Test Driven Development: A practical guide*. Upper Saddle River, N.J.: Prentice Hall, 2003.
- Beck, Kent. *Extreme Programming Explained: Embrace change*. Boston: Addison-Wesley, 2000.
- . *Test Driven Development*. Reading, Mass.: Addison-Wesley, 2003.
- Beck, Kent, and Martin Fowler. *Planning Extreme Programming*. Reading, Mass.: Addison-Wesley, 2000.
- Beedle, Mike, et al. "SCRUM: A Pattern Language for Hyperproductive Software Development." In Neil Harrison et al. (Eds.), *Pattern Languages of Program Design 4*. Addison-Wesley: 1999, pp. 637–651.
- Boehm, Barry. "A Spiral Model of Development and Enhancement." *IEEE Computer* 28, no. 5 (May 1988): 61–72.
- . *Software Engineering Economics*. Englewood Cliffs, N.J.: Prentice-Hall, 1981.
- Bower, G. H., J. B. Black, and T. J. Turner. "Scripts in Memory for Text." *Cognitive Psychology* 11 (1979): 177–220.
- Carroll, John M. "Making Use a Design Representation." *Communications of the ACM* 37, no. 12 (December 1994): 29–35.
- . *Making Use: Scenario-based design in human-computer interaction*. Cambridge, Mass.: The MIT Press, 2000.
- . "Making use is more than a matter of task analysis." *Interacting with Computers* 14, no. 5 (2002): 619–627.

- Carroll, John M., Mary Beth Rosson, George Chin Jr., and Jürgen Koenemann. "Requirements Development in Scenario-Based Design." *IEEE Transactions on Software Engineering* 24, no. 12 (December 1998): 1156–1170.
- Cirillo, Francesco. "XP: Delivering the Competitive Edge in the Post-Internet Era." At www.communications.xplabs.com/paper2001-3.html. XP Labs, 2001.
- Cockburn, Alistair. *Writing Effective Use Cases*. Upper Saddle River, N.J.: Addison-Wesley, 2001.
- Cockburn, Alistair, and Laurie L. Williams. "The Costs and Benefits of Pair Programming." In Giancarlo Succi and Michele Marchesi (Eds.), *Extreme Programming Examined*. Upper Saddle River, N.J.: Addison-Wesley, 2001.
- Cohn, Mike. "The Upside of Downsizing." *Software Test and Quality Engineering* 5, no. 1 (January 2003): 18–21.
- Constantine, Larry. "Cutting Corners." *Software Development* (February 2000).
- . "Process Agility and Software Usability: Toward lightweight and usage-centered design." *Information Age* (August-September 2002).
- Constantine, Larry L., and Lucy A. D. Lockwood. *Software for Use: A practical guide to the models and methods of usage-centered design*. Reading, Mass.: Addison-Wesley, 1999.
- . "Usage-Centered Engineering for Web Applications." *IEEE Software* 19, no. 2 (March/April 2002): 42–50.
- Cooper, Alan. *The Inmates Are Running the Asylum*. Indianapolis: SAMS, 1999.
- Cusumano, Michael A., and Richard W. Selby. *Microsoft Secrets: How the world's most powerful software company creates technology, shapes markets, and manages people*. New York: The Free Press, 1995.
- Davies, Rachel. "The Power of Stories." XP 2001. Sardinia, 2001.
- Djajadiningrat, J.P., W. W. Gaver and J. W. Frens. "Interaction Relabelling and Extreme Characters: Methods for exploring aesthetic interactions." *Symposium on Designing Interactive Systems 2000*, 2000: 66–71.
- Fowler, Martin. "The Almighty Thud." *Distributed Computing* (November 1997).
- Fowler, Martin, et al. *Refactoring: Improving the design of existing code*, Reading, Mass.: Addison-Wesley, 1999.
- Gilb, Tom. *Principles of Software Engineering Management*. Reading, Mass.: Addison-Wesley, 1988.
- Guindon, Raymonde. "Designing the Design Process: Exploiting opportunistic thoughts." *Human-Computer Interaction* 5, 1990.

- Grudin, Jonathan, and John Pruitt. "Personas, Participatory Design and Product Development: An Infrastructure for Engagement." In Thomas Binder, Judith Gregory, and Ina Wagner (Eds.), *Participation and Design: Inquiring into the politics, contexts and practices of collaborative design work, Proceedings of the Participatory Design Conference 2002*: 2002: 144–161.
- IEEE Computer Society. *IEEE Recommended Practice for Software Requirements Specifications*. New York, 1998.
- Jacobson, Ivar. *Object-Oriented Software Engineering*. Upper Saddle River, N.J.: Addison-Wesley, 1992.
- Jacobson, Ivar, Grady Booch, and James Rumbaugh. *The Unified Software Development Process*. Reading, Mass: Addison-Wesley, 1999.
- Jeffries, Ron. "Essential XP: Card, Conversation, and Confirmation." *XP Magazine* (August 30, 2001).
- Jeffries, Ron, Ann Anderson, and Chet Hendrickson. *Extreme Programming Installed*. Boston: Addison-Wesley, 2000.
- Kensing, Finn, and Andreas Munk-Madsen. "PD: Structure in the Toolbox." *Communications of the ACM* 36, no. 6 (June 1993): 78–85.
- Kovitz, Ben L. *Practical Software Requirements: A manual of content and style*. Greenwich, Conn.: Manning, 1999.
- Kuhn, Sarah, and Michael J. Muller. "Introduction to the Special Section on Participatory Design." *Communications of the ACM* 36, no. 6 (June 1993): 24–28.
- Lauesen, Soren. *Software Requirements: Styles and techniques*. London: Addison-Wesley, 2002.
- Lundh, Erik, and Martin Sandberg. "Time Constrained Requirements Engineering with Extreme Programming: An experience report." In Armin Eberlein and Julio Cesar Sampaio do Prado Leite (Eds.), *Proceedings of the International Workshop on Time Constrained Requirements Engineering*, 2002.
- Newkirk, James, and Robert C. Martin. *Extreme Programming in Practice*. Upper Saddle River, N.J.: Addison-Wesley, 2001.
- Parnas, David L., and Paul C. Clements. "A Rational Design Process: How and why to fake it." *IEEE Transactions on Software Engineering* 12, no. 2 (February 1986): 251–7.
- Patton, Jeff. "Hitting the Target: Adding interaction design to agile software development." Conference on Object Oriented Programming Systems Languages and Applications (OOPSLA 2002). New York: ACM Press, 2002.
- Poppendieck, Tom. *The Agile Customer's Toolkit*. In Larry L. Constantine (Ed.), *Proceedings of forUSE 2003*. Rowley, Mass.: Ampersand Press: 2003.

- Potts, Colin, Kenji Takahashi, and Annie I. Antón. "Inquiry-Based Requirements Analysis." *IEEE Software* 11, no. 2 (March/April 1994): 21–32.
- Robertson, Suzanne and James Robertson. *Mastering the Requirements Process*. Reading, Mass.: Addison-Wesley, 1999.
- Schuler, Douglas, and Aki Namioka (Eds.). *Participatory Design: Principles and practices*. Hillsdale, N.J.: Erlbaum, 1993.
- Schwaber, Ken, and Mike Beedle. *Agile Software Development with Scrum*. Upper Saddle River, N.J.: Prentice Hall, 2002.
- Stapleton, Jennifer. *DSDM: Business Focused Development*. Reading, Mass.: Addison-Wesley, 2003.
- Swartout, William, and Robert Balzer. "On the Inevitable Intertwining of Specification and Implementation." *Communications of the ACM* 25, no. 7 (July 1982): 438–440.
- Wagner, Larry. "Extreme Requirements Engineering." *Cutter IT Journal* 14, no. 12 (December 2001).
- Wake, William C. *Extreme Programming Explored*. Reading, Mass.: Addison-Wesley, 2002.
- . "INVEST in Good Stories, and SMART Tasks." At www.xp123.com, 2003a.
- . *Refactoring Workbook*. Reading, Mass.: Addison-Wesley, 2003b.
- Weidenhaupt, Klaus, Klaus Pohl, Matthias Jarke, and Peter Haumer. "Scenarios in System Development: Current practice." *IEEE Software* 15, no. 2 (March/April 1998): 34–45.
- Wieggers, Karl E. *Software Requirements*. Redmond, Wash.: Microsoft Press, 1999.
- Williams, Marian G., and Vivienne Begg. "Translation between software designers and users." *Communications of the ACM* 36, no. 6 (June 1993): 102–3.

Websites

www.agilealliance.com
www.controlchaos.com
www.foruse.com
www.mountangoatsoftware.com
www.userstories.com
www.xprogramming.com
www.xp123.com